

Aerodynamic Simulations in Julia with Trixi.jl

Joint Workshop DLR-RWTH

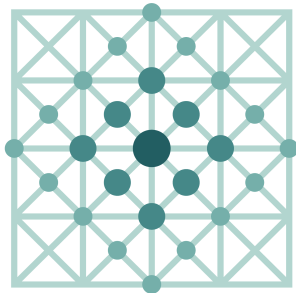
Daniel Doebling, Manuel Torrilhon¹

in collaboration with

Arpit Babbar², Jesse Chan³,
Ahmad Peyvan⁴, and Andrew Winters⁵

¹ACoM, RWTH Aachen University

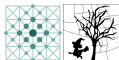
²JGU Mainz ³Oden Institute, UT Austin ⁴Applied Mathematics, Brown ⁵TIMA, Linköping



First Things First

Acknowledgements:

- **Arpit Babbar:**
Drag, Lift, Pressure and Friction Coefficients
- **Jesse Chan & Ahmad Peyvan:**
Parabolic adaptive mesh refinement (AMR),
particularly mortars
- **Andrew Winters:**
Boundary identification for Gmsh/Abaqus meshes



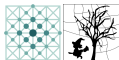
First Things First

Acknowledgements:

- **Arpit Babbar:**
Drag, Lift, Pressure and Friction Coefficients
- **Jesse Chan & Ahmad Peyvan:**
Parabolic adaptive mesh refinement (AMR),
particularly mortars
- **Andrew Winters:**
Boundary identification for Gmsh/Abaqus meshes

Reproducibility:

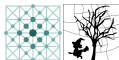
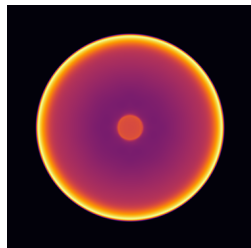
- Public repository on GitHub:
→ <https://github.com/DanielDoehring/paper-2025-perrk>



About Trixi.jl

Trixi.jl → <https://github.com/trixi-framework/Trixi.jl>

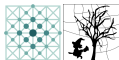
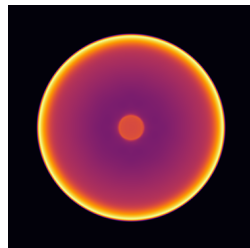
- is a **Julia** package for simulation of *compressible* (Euler, NSF, MHD) and SW flows



About Trixi.jl

Trixi.jl → <https://github.com/trixi-framework/Trixi.jl>

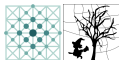
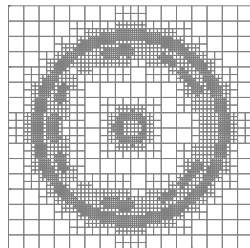
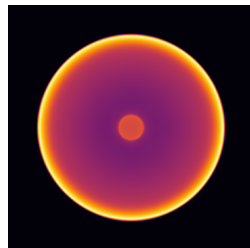
- is a **Julia** package for simulation of *compressible* (Euler, NSF, MHD) and SW flows
- implements the (entropy-stable) Discontinuous Galerkin Spectral Element (DG-SEM) method



About Trixi.jl

Trixi.jl → <https://github.com/trixi-framework/Trixi.jl>

- is a **Julia** package for simulation of *compressible* (Euler, NSF, MHD) and SW flows
- implements the (entropy-stable) Discontinuous Galerkin Spectral Element (DG-SEM) method
- is a **grid-based** solver with adaptive mesh refinement (AMR)



About Trixi.jl

History:



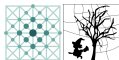
FLUXO



- FLEXI (Stuttgart; Pls: *Munz, Beck*)
- FLUXO (Cologne; Pl: *Gassner, Winters*)
- Trixi (Cologne; Pls: *Schlottke-Lakemper, Gassner, Ranocha*)

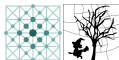
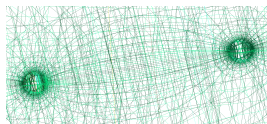
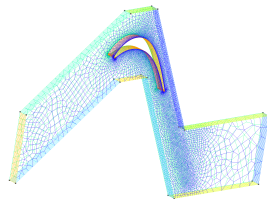
Current participating groups:

- **Germany:** Aachen, Augsburg, Cologne, Hamburg, Mainz
- **US:** Austin, Brown
- **Spain:** Madrid
- **Sweden:** Linköping
- **India:** Bangalore, Delhi



Re-use existing Meshes

Mesh generation is a non-trivial task, especially for "true", i.e., non-extruded 3D meshes!

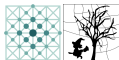
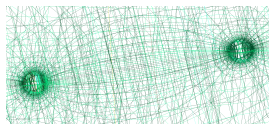
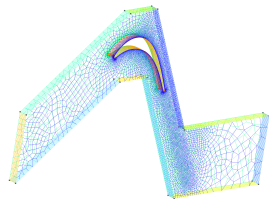


Re-use existing Meshes

Mesh generation is a non-trivial task, especially for "true", i.e., non-extruded 3D meshes!

Goals: Be able to use existing

- Meshes in standardized format (.inp)
- Tools for mesh generation $\Rightarrow$ Gmsh



Re-use existing Meshes

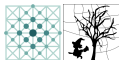
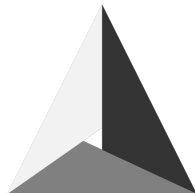
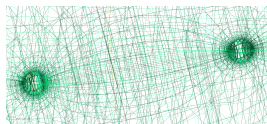
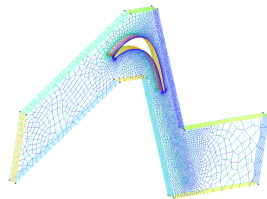
Mesh generation is a non-trivial task, especially for "true", i.e., non-extruded 3D meshes!

Goals: Be able to use existing

- Meshes in standardized format (.inp)
- Tools for mesh generation $\Rightarrow$ Gmsh

While **maintaining features** like

- Adaptive mesh refinement (AMR)
- Parallelization
- Visualization
- Restarting



Re-use existing Meshes

Mesh generation is a non-trivial task, especially for "true", i.e., non-extruded 3D meshes!

Goals: Be able to use existing

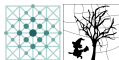
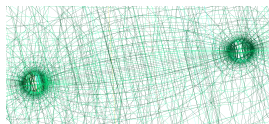
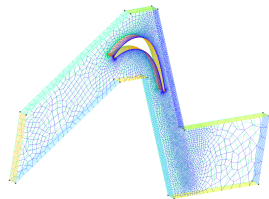
- Meshes in standardized format (.inp)
- Tools for mesh generation $\Rightarrow$ Gmsh

While **maintaining features** like

- Adaptive mesh refinement (AMR) ✓
- Parallelization ✓
- Visualization ✓
- Restarting ✓

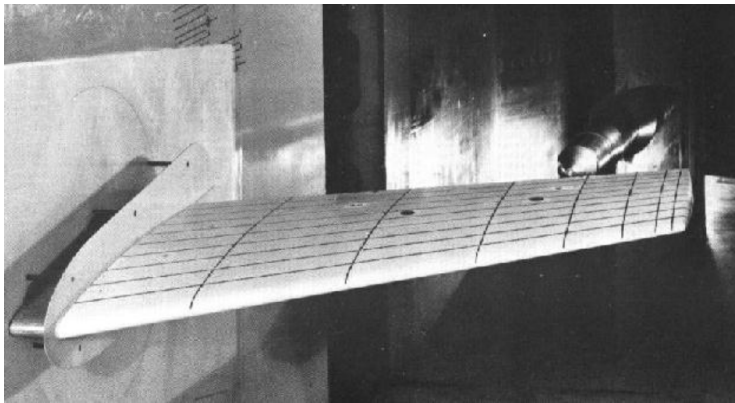
$\Rightarrow$ Use `P4est.jl` wrapper for `p4est` library

$\Rightarrow$ `T8Code.jl` wrapper for `t8code` library

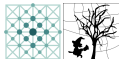


Case Study: ONERA M6 Wing

- $Ma_\infty = 0.84$, $AoA = 3.06^\circ$, $(Re_\infty = 11.72 \cdot 10^6)$
- Mesh with $\sim 2.95 \cdot 10^5$ cells ($\mathcal{O}(2)$ FV RANS-ready)
⇒ Available for download from [NASA webpage](#)



Picture taken from <https://www.grc.nasa.gov/www/wind/valid/m6wing/m6wing.html>



Case Study: ONERA M6 Wing

Initial Condition & Farfield Boundary Condition

- $Ma_{\infty} = 0.84$, $AoA = 3.06^{\circ}$

```
using Trixi

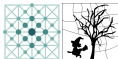
@inline function initial_condition(x, t, equations)
    # Non-dimensionalized (for gas with  $\gamma = 1.4$ ,  $\mathcal{R} = 1$ )
    rho = 1.4

    v1 = 0.8388023121403883 #  $0.84 \cdot \cos(3.06^{\circ})$ 
    v2 = 0.0448406193973588 #  $0.84 \cdot \sin(3.06^{\circ})$ 
    v3 = 0.0

    p = 1.0

    prim = SVector{rho, v1, v2, v3, p}
    return prim2cons(prim, equations)
end

bc_farfield = BoundaryConditionDirichlet(initial_condition)
```



Case Study: ONERA M6 Wing

Mesh-Import:

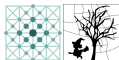
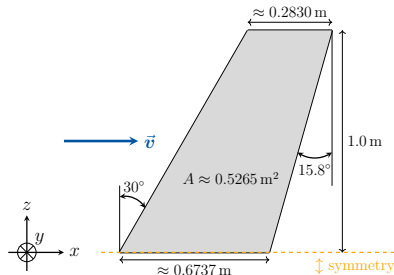
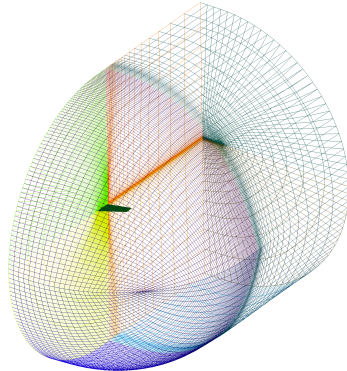
- 1 Merge 4-block mesh
- 2 Sanitize degenerated cells
- 3 Export .msh via Gmsh to .inp

Code:

```
mesh_file = "m6wing-sanitized.inp"

boundaries = [:Symmetry, # z = 0 boundary
              :FarField, # Freestream
              :BottomWing, :TopWing]

mesh = P4estMesh{3}(mesh_file,
                    boundary_symbols = boundaries)
```



Case Study: ONERA M6 Wing

Equations, Solver & Boundary Conditions

```
gamma = 1.4 # Ratio of specific heats  
equations = CompressibleEulerEquations3D(gamma)
```

Case Study: ONERA M6 Wing

Equations, Solver & Boundary Conditions

```
gamma = 1.4 # Ratio of specific heats
equations = CompressibleEulerEquations3D(gamma)

volume_integral = VolumeIntegralFluxDifferencing(flux_ranocha)

# About  $40 \cdot 10^6$  degrees of freedom
solver = DGSEM(polydeg = 2,
               surface_flux = flux_lax_friedrichs,
               volume_integral = volume_integral)
```

Case Study: ONERA M6 Wing

Equations, Solver & Boundary Conditions

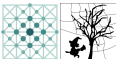
```
gamma = 1.4 # Ratio of specific heats
equations = CompressibleEulerEquations3D(gamma)

volume_integral = VolumeIntegralFluxDifferencing(flux_ranocha)

# About  $40 \cdot 10^6$  degrees of freedom
solver = DGSEM(polydeg = 2,
               surface_flux = flux_lax_friedrichs,
               volume_integral = volume_integral)

boundary_conditions = Dict(:Symmetry => bc_symmetry,
                          :FarField => bc_farfield, # Weakly enforced initial condition
                          :BottomWing => boundary_condition_slip_wall,
                          :TopWing => boundary_condition_slip_wall)

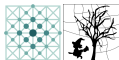
semi = SemidiscretizationHyperbolic(mesh, equations, initial_condition, solver,
                                   boundary_conditions = boundary_conditions)
```



Case Study: ONERA M6 Wing

ODE & Multirate Time Integration

```
tspan = (0.0, 6.0)  
ode = semidiscretize(semi, tspan)
```

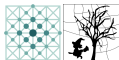
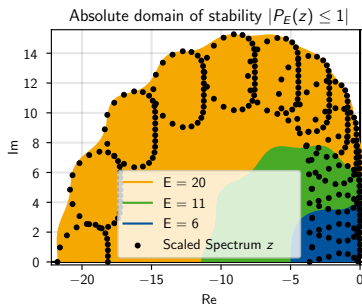
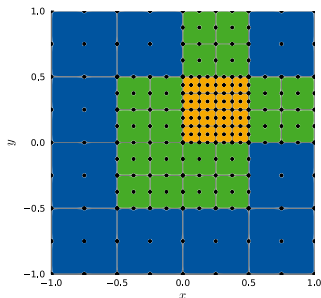


Case Study: ONERA M6 Wing

ODE & Multirate Time Integration

```
tspan = (0.0, 6.0)  
ode = semidiscretize(semi, tspan)
```

Multirate time-integration by using **stabilized/optimized** schemes in regions with higher characteristic speeds (stricter CFL)



Case Study: ONERA M6 Wing

ODE & Multirate Time Integration

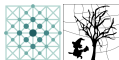
Partition ODE according to characteristic speeds $a_i := \min_j \frac{\Delta x_j}{|\mu_j(\mathbf{u}_i)|}$

$$\mathbf{U}'(t) = \begin{pmatrix} \mathbf{U}^{(1)}(t) \\ \vdots \\ \mathbf{U}^{(R)}(t) \end{pmatrix}' = \begin{pmatrix} \mathbf{F}^{(1)}(t, \mathbf{U}^{(1)}(t), \dots, \mathbf{U}^{(R)}(t)) \\ \vdots \\ \mathbf{F}^{(R)}(t, \mathbf{U}^{(1)}(t), \dots, \mathbf{U}^{(R)}(t)) \end{pmatrix} = \mathbf{F}(t, \mathbf{U}(t))$$

and employ different RKMs

$$\mathbf{K}_i^{(r)} = \mathbf{F}^{(r)} \left(t_n + c_i \Delta t, \mathbf{U}_n + \Delta t \sum_{j=1}^S \sum_{k=1}^R a_{i,j}^{(k)} \mathbf{K}_j^{(k)} \right),$$
$$\mathbf{U}_{n+1} = \mathbf{U}_n + \Delta t \sum_{i=1}^S b_i \sum_{k=1}^R \mathbf{K}_i^{(r)}$$

optimized for the spectrum $\sigma \left(\frac{\partial \mathbf{F}}{\partial \mathbf{U}} \Big|_{\mathbf{U}_0} \right)$.



Case Study: ONERA M6 Wing

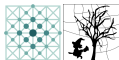
ODE & Multirate Time Integration

- Multirate Relaxation Runge-Kutta for Entropy Conservation/Stability

```
newton = RelaxationSolverNewton(max_iterations = 5, root_tol = 1e-12)

ode_alg = PairedExplicitRelaxationRK3Multi(stages, path, dt_ratios;
                                           relaxation_solver = newton)

sol = solve(ode, ode_alg);
```



Case Study: ONERA M6 Wing

ODE & Multirate Time Integration

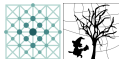
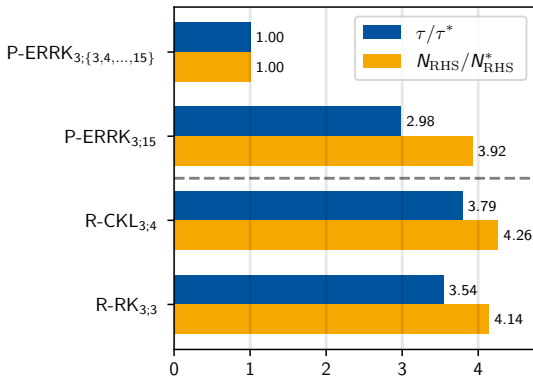
- Multirate Relaxation Runge-Kutta for Entropy Conservation/Stability

```
newton = RelaxationSolverNewton(max_iterations = 5, root_tol = 1e-12)

ode_alg = PairedExplicitRelaxationRK3Multi(stages, path, dt_ratios;
                                           relaxation_solver = newton)

sol = solve(ode, ode_alg);
```

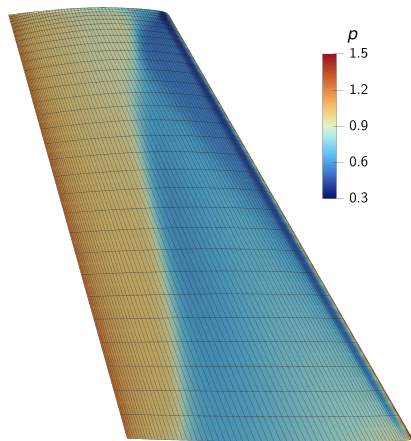
Runtime and RHS Evaluations



Case Study: ONERA M6 Wing

Results

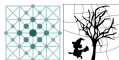
- $k = 2$, Rusanov, Flux-Diff. (no limiting needed!)



Lift-Coefficient

$$C_L := \frac{L(p)}{0.5\rho_\infty U_\infty^2 A}$$

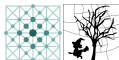
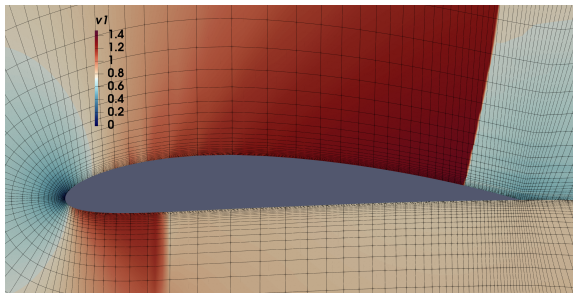
Trixi.jl, Projected A: $C_L \approx 0.2951$
SU2, Computed A: $C_L \approx 0.2861$
(-3%)



AMR with NACA 4412 Airfoil

Inviscid transonic flow

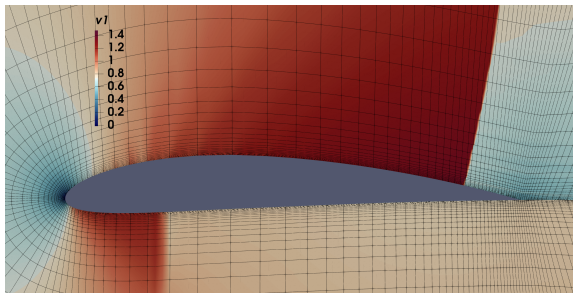
- NASA TMR mesh



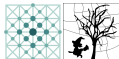
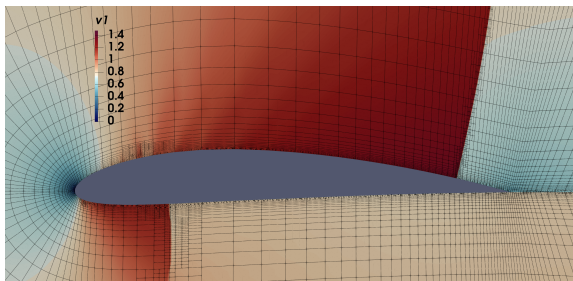
AMR with NACA 4412 Airfoil

Inviscid transonic flow

- NASA TMR mesh



- NASA TMR mesh
+ 3-Level AMR



Viscous, Laminar SD7003 Airfoil

Unsteady Flow

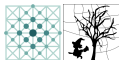
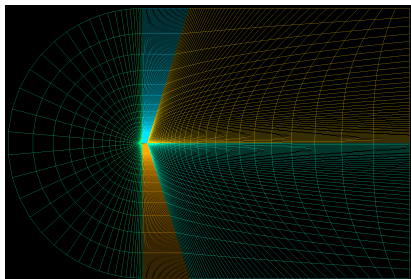
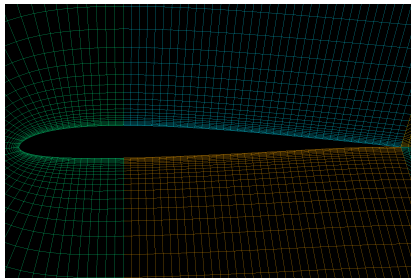
- $Re = 10^4, Ma = 0.2$
- $Pr = 0.72, \gamma = 1.4$
- $\alpha = 4^\circ$

Mesh:

- $x_{\min} = -20, x_{\max} = 60,$
 $y_{\min} = -20, y_{\max} = 20$
- 7605 elements

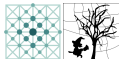
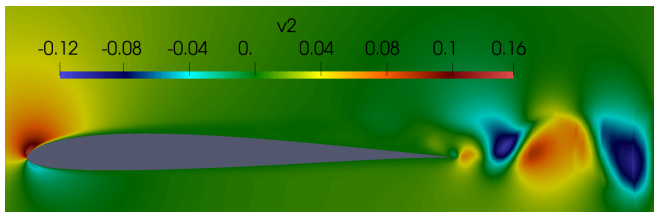
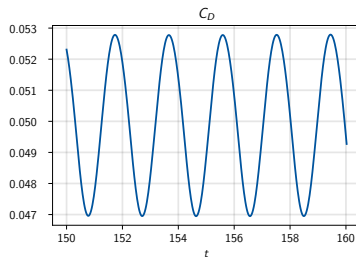
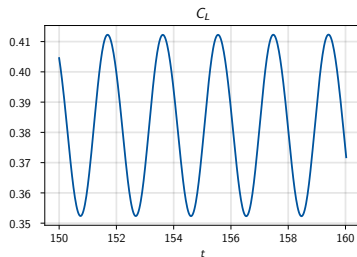
Discretization:

- Space: 4th-order DGSEM
- Entropy-stable flux-differencing
- Time: 4th-order Multirate P-ERRK



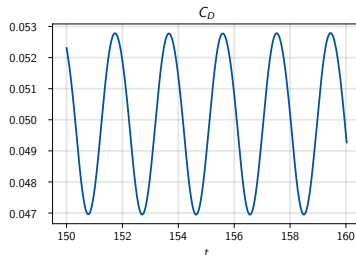
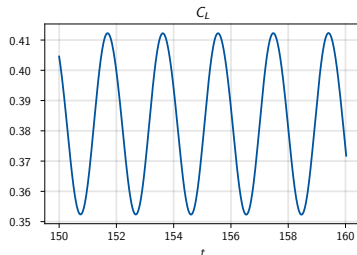
Viscous, Laminar SD7003 Airfoil

Unsteady Flow



Viscous, Laminar SD7003 Airfoil

Unsteady Flow

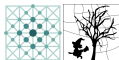


Source	$\overline{C}_L = \overline{C}_{L,p}$	$\overline{C}_D = \overline{C}_{D,p} + \overline{C}_{D,\mu}$
Trixi.jl	0.3827	0.04995
Vermeire ^a	0.3841	0.04990
Uranga et al. ^b	0.3755	0.04978
López-Morales et al. ^c	0.3719	0.04940

^aB. C. Vermeire, JCP, 2019.

^bA. Uranga et al., Int. J. Num Methods Engineering, 2011

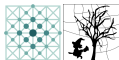
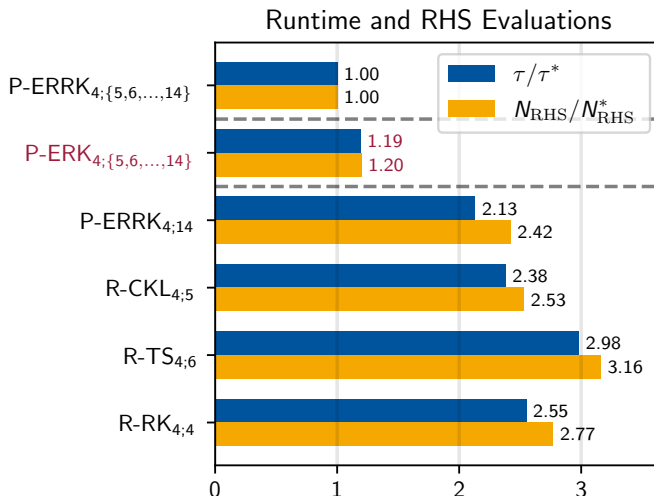
^cM. López-Morales et al., 32nd AIAA AAC, 2014



Viscous, Laminar SD7003 Airfoil

Unsteady Flow

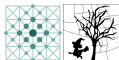
Entropy-Stable Multirate Time-Integration ($p = 4$)



Conclusion & Outlook

Summary:

- Import of existing quadrilateral meshes in `.inp` format
- Ready for participation in High-Order/Fidelity CFD workshops
- Non-intrusive, EC/ES optimized multirate time integration



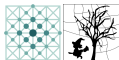
Conclusion & Outlook

Summary:

- Import of existing quadrilateral meshes in `.inp` format
- Ready for participation in High-Order/Fidelity CFD workshops
- Non-intrusive, EC/ES optimized multirate time integration

Outlook:

- Higher order (curved) quad/hex elements from Gmsh/
in `.inp` format
- Beyond quads/hexes with `T8Code.jl`
→ <https://github.com/DLR-AMR/T8code.jl>



Thank you for your attention!

Questions?



doehring@acom.rwth-aachen.de



<https://www.acom.rwth-aachen.de>

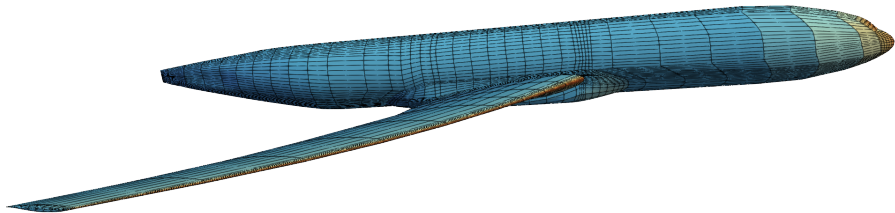


<https://github.com/DanielDoehring>



<https://doi.org/10.48550/arXiv.2507.04991>

Reproducibility Repository: <https://github.com/DanielDoehring/paper-2025-perrk>



NASA Common Research Model, $Ma_{\infty} = 0.85$, $Re_{\infty} = 200 \cdot 10^6$

